Operating System & Advanced Mobile Lab 2

Report – Simple scheduling

By Sebastien BERTIL SOUCHET (32239360) & Ewem EMERAUD (32239362)

Due Date: 2025/11/17

1. Introduction

This project implements a simple CPU scheduling simulator in C using **processes** and **inter-process communication (IPC)**. It focuses on building a correct and efficient **round-robin scheduler** that manages multiple child processes, simulating CPU and I/O bursts.

Goals

- Create a parent process that spawns 10 child processes.
- Simulate process execution with dynamic **CPU bursts** and optional **I/O bursts**.
- Implement a **round-robin scheduling policy** with a configurable time quantum and timer tick.
- Manage **run-queue** (ready processes) and **wait-queue** (processes performing I/O).
- Use **message queues (msgget, msgsnd, msgrcv)** for IPC to notify processes about CPU time and I/O requests.
- Track **waiting time** and **remaining time quantum** for all processes.
- Log detailed scheduling operations to schedule_dump.txt for analysis.

What it supports

- Parent process is acting as the **OS scheduler**.
- 10 child processes simulating independent user programs.
- Periodic timer interrupts using setitimer and SIGALRM.
- Round-robin scheduling with preemption and I/O handling.
- Unix-like systems with POSIX IPC and signal handling.
- Logging of run-queue and wait-queue states at each tick.

2. Instructions to Build the Program

Requirements

- GCC (or Clang)
- Linux / Unix-like environment
- POSIX threads (pthreads) available by default on Linux

Files

- **sched.c:** main program, process creation, round-robin scheduler, parent & child loops, logging.

- **msg.h**: message queue structure and constants.
- **Makefile**: build automation for compiling the program.

Build

Run:

make

This produces the binary :

./os_proj1

To clean build artifacts:

make clean

To clean everything (artifacts & binaries):

make fclean

To rebuild from scratch:

make re

Example run

./ os_proj1

3. How the Program Works

General Flow

Initialization At startup, the parent process creates a System V message queue and then forks NCHILD child processes. It initializes two circular queues:

- the run queue, which holds processes ready to execute,
- the wait queue, which holds processes currently performing an I/O burst.

Each PCB stores the pid of the child, its I/O state, remaining quantum, and accumulated waiting time. A periodic timer is then configured using setitimer, generating SIGALRM signals at fixed intervals. Each signal represents one scheduling tick.

Signal-driven scheduling Each tick triggers a signal handler that increments counters. The parent's main loop processes all pending ticks by executing one full scheduling step per tick.

Scheduler Tick Logic

1. Advancing I/O bursts Every tick, all processes in the wait queue have their remaining I/O time decreased. When an I/O burst completes, the process is removed from the wait queue and returned to the run queue as READY.

2. Updating waiting time All processes currently waiting in the run queue increment their waiting-time counter.

3. Handling the current running process If a process is currently on the CPU:

- If it initiates an I/O burst, it is immediately descheduled.
- Otherwise, its remaining quantum is decremented. When the quantum expires, the process is preempted and placed at the end of the run queue.

4. Selecting the next process If the CPU is idle and the run queue is non-empty, the scheduler dequeues the next READY process, assigns it a fresh time quantum, and sends it a message indicating it may run for one tick.

5. Handling child I/O requests Child processes notify the parent when they begin an I/O burst. Upon receiving such a request, the scheduler:

- identifies the sending process,
- removes it from the CPU or from the run queue,
- inserts it into the wait queue with its I/O burst duration.

6. Logging For the first part of the simulation, the scheduler logs CPU activity, remaining quantum, and the contents of both queues. This produces a trace showing the full evolution of the scheduling state.

Child Process Behavior

Each child runs in an infinite loop. It blocks while waiting for a message from the parent, which grants it one CPU tick. Upon receiving the message:

- it decrements its current CPU burst counter,
- when the burst ends, it generates a random I/O burst duration, sends a message to the parent requesting I/O, and starts a new pair of CPU and I/O bursts.

The children therefore alternate between CPU bursts and I/O bursts until the parent terminates the simulation.

Parent Loop and Termination

The parent continues processing ticks until a predefined maximum number of ticks is reached. During this time, it repeatedly:

- handles timer-driven scheduling,
- moves processes between CPU, RUNNING, and WAITING states,
- services all I/O requests,
- updates queues, statistics, and logs.

At the end of the simulation:

- the parent sends a termination signal to all children,
- waits for their exit,
- removes the message queue,
- and closes the log file.

Execution Path Summary

- CPU idle → the next READY process is selected.
- Child requests I/O → it is moved to the wait queue.
- I/O burst completes → the process returns to the run queue.
- Quantum expires → process is preempted and placed at the end of the run queue.
- Tick occurs → all scheduling logic advances by one unit.
- Simulation ends → children are cleaned up and IPC resources removed.

Why It's Correct & Efficient

Correctness The scheduler is deterministic and tick-driven, avoiding race conditions. Each child receives only the messages intended for it. State transitions between READY, RUNNING, and WAITING are explicitly managed using two queues. No process is lost or left in an undefined state. The parent alone performs scheduling, ensuring coherency.

Efficiency Children never busy-wait; they block on message reception. Message passing is cheap and predictable. The scheduler's complexity is bounded by the fixed number of children. Signal-driven ticking models a real preemptive OS while keeping overhead low.

4. Example Usage

```
┌─(~/DKU/OS/2025_os_proj1)──────────────────────────────────────────────────────────────────────────────────────────┐
└─(14:32:40 on 32239360_sebastien_32239362_ewen ✹ ⋆)──> ./os_proj1
[SUB-PROCESS 18207] Create
[SUB-PROCESS 18208] Create
[SUB-PROCESS 18209] Create
[SUB-PROCESS 18210] Create
[SUB-PROCESS 18211] Create
[SUB-PROCESS 18212] Create
[SUB-PROCESS 18213] Create
[SUB-PROCESS 18214] Create
[SUB-PROCESS 18215] Create
[SUB-PROCESS 18216] Create
[INFO] Running ...
[INFO] For more information check log file : schedule_dump.txt
[SUB-PROCESS 18207] Kill
[SUB-PROCESS 18208] Kill
[SUB-PROCESS 18209] Kill
[SUB-PROCESS 18210] Kill
[SUB-PROCESS 18211] Kill
[SUB-PROCESS 18212] Kill
[SUB-PROCESS 18213] Kill
[SUB-PROCESS 18214] Kill
[SUB-PROCESS 18215] Kill
[SUB-PROCESS 18216] Kill
[INFO] End
```

```
┌─(~/DKU/OS/2025_os_proj1)──────────────────────────────────────────────────────────────────────────────────────────┐
└─(14:32:40 on 32239360_sebastien_32239362_ewen ✹ ⋆)──> tail -n25 schedule_dump.txt

(time 6995) process pid=18213 gets cpu time, remaining time-quantum=3
  run-queue: [0(pid=18207), 8(pid=18215), 1(pid=18208), 9(pid=18216), 5(pid=18212), 7(pid=18214),
3(pid=18210), 4(pid=18211), 2(pid=18209)]
  wait-queue: []

(time 6996) process pid=18213 gets cpu time, remaining time-quantum=2
  run-queue: [0(pid=18207), 8(pid=18215), 1(pid=18208), 9(pid=18216), 5(pid=18212), 7(pid=18214),
3(pid=18210), 4(pid=18211), 2(pid=18209)]
  wait-queue: []

(time 6997) process pid=18213 gets cpu time, remaining time-quantum=1
  run-queue: [0(pid=18207), 8(pid=18215), 1(pid=18208), 9(pid=18216), 5(pid=18212), 7(pid=18214),
3(pid=18210), 4(pid=18211), 2(pid=18209)]
  wait-queue: []

(time 6998) process pid=18207 gets cpu time, remaining time-quantum=5
  run-queue: [8(pid=18215), 1(pid=18208), 9(pid=18216), 5(pid=18212), 7(pid=18214), 3(pid=18210),
4(pid=18211), 2(pid=18209), 6(pid=18213)]
  wait-queue: []

(time 6999) process pid=18207 gets cpu time, remaining time-quantum=4
  run-queue: [8(pid=18215), 1(pid=18208), 9(pid=18216), 5(pid=18212), 7(pid=18214), 3(pid=18210),
4(pid=18211), 2(pid=18209), 6(pid=18213)]
  wait-queue: []

(time 7000) process pid=18207 gets cpu time, remaining time-quantum=3
  run-queue: [8(pid=18215), 1(pid=18208), 9(pid=18216), 5(pid=18212), 7(pid=18214), 3(pid=18210),
4(pid=18211), 2(pid=18209), 6(pid=18213)]
  wait-queue: []
```

6. Usage of AI

I used an AI assistant to help refine the **print of the logs** so the final output is clearer and more readable (aligned columns, consistent spacing).

Concretely, the AI helped me:

- Choose a **format output** for the log
- Improve the **tabular layout** (line breaks, column alignment, compact grouping).

Scope & attribution:

- The AI was used **only** for formatting of the logs
- All **core logic** was designed and implemented by us